

CSC1005: Introduction to Computer Engineering

Programming and Applications

Assignment 2

Assignment description:

This assignment will be worth 8% of the final grade.

You should write your code for each question in a .py file (please name it using the question name, e.g. q1.py). Please pack all your .py files into a single .zip file, name it using your student ID (e.g. if your student ID is 123456, then the file should be named as 123456.zip), and then submit the .zip file via Blackboard.

Please create a Word document that provides a detailed explanation of the code for each question. This document should clearly outline the logic behind your code, specify the expected inputs, and describe the anticipated outputs. Make sure the explanations are easy to understand and provide enough context for anyone reviewing your work. Include this Word document in the .zip file along with your code.

Please be aware that the teaching assistant may ask you to explain your code to verify that it was written by you. Additionally, your submission may be checked for similarities with other students' work using Blackboard to ensure academic integrity.

This assignment is due on 5:00PM, 14 Nov (Friday). For each day of late submission, you will lose 10% of your mark in this assignment. If you submit more than three days later than the deadline, you will receive zero in this assignment.

If you have any questions about Assignment 2 (AS2), please contact the TA for AS2, LIAO Huanxin, at 221019077@link.cuhk.edu.cn

Question 1: Reversible Prime Square (15% of this assignment)

A *reversible prime square* is a number that is both a perfect square and a prime when reversed. A perfect square is a number that can be expressed as the product of an integer multiplied by itself. For example, 169 is a perfect square (the square of 13), and becomes 961 when reversed, which is a prime. Write a program that displays the first 50 reversible prime squares. Display 10 numbers per line, aligned properly.

Question 2: Mirror Words (15% of this assignment)

A *mirror word* is a word where the first half is the same as the reverse of the second half. For example, "deified" and "redder" are mirror words.

Write a function that takes a sentence as input and returns all the mirror words in a list. If a word has an odd number of letters, ignore the middle letter when checking if it is a mirror word.

Example:

```
find_mirror_words("level radar noon civic kayark madam")
```

Return of your function:

```
['level', 'radar', 'noon', 'civic', 'madam']
```

Question 3: Anagram Grouping (25% of this assignment)

Two words are anagrams if they contain the same letters in different orders, like "listen" and "silent."

Write a function that takes a list of words and groups all anagrams together. Each group of anagrams should be presented as a list, and all groups should be returned in a list.

Example:

```
group_anagrams(["listen", "silent", "enlist", "rat", "tar", "god", "dog", "evil", "vile", "veil"])
```

Return of your function:

```
[['listen', 'silent', 'enlist'], ['rat', 'tar'], ['god', 'dog'], ['evil', 'vile', 'veil']]
```

Question 4: Rotated Words (20% of this assignment)

A *rotated word* is a word that can be transformed into another by shifting its letters circularly. For example, "hello" can be rotated to become "ohell", "lohel", etc.

Write a function that takes a list of words and groups words that are rotations of each other into sublists. Each group should contain all rotations of a particular word, but each word should appear only once across all groups.

Example:

```
group_rotated_words(["hello", "ohell", "lohel", "elloh", "world", "dlrow", "orlwd"])
```

Return of your function:

```
[['hello', 'ohell', 'lohel', 'elloh'], ['world', 'ldwor', 'orldw']]
```

Question 5: Character Frequency Analysis (25% of this assignment)

Given a string, you need to analyze the frequency of each character. However, instead of a simple count, you should group characters by their frequencies.

Write a function that takes a string as input and returns a dictionary where the keys are frequencies, and the values are lists of characters with that frequency. Sort the characters in each list alphabetically.

Example:

```
character_frequency("aabbccdddddcc")
```

Return of your function:

```
{2: ['a', 'b'], 3: ['c'], 5: ['d']}
```